

44.2 Backfitting and concavity

Section 44.1 reduced the additive model to one penalized regression with $d = p + \sum_j d_j$ columns. When d is a few dozen, solving Eq. 44.1.6 directly is right and this section is only a change of view. When d runs into the thousands — one coefficient per district, one per subject — or when a term's smoother is not given by a basis at all, an older idea takes over: fit each term in turn to what the others have left behind, and go round again.

44.2.1 The algorithm

Write $\mathbf{S}_j$ for the smoother of the j th term, the linear operator that fits that term alone to a response vector:

$$\mathbf{S}_j = \mathbf{Z}_j(\mathbf{Z}_j^\top \mathbf{Z}_j + \lambda_j \mathbf{P}_j)^{-1} \mathbf{Z}_j^\top. \quad (44.2.1)$$

The **partial residual** for term j is what the response has left once every other part of the model is subtracted, $\mathbf{r}_j = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}} - \sum_{k \neq j} \hat{\mathbf{f}}_k$; backfitting replaces $\hat{\mathbf{f}}_j$ by $\mathbf{S}_j \mathbf{r}_j$ and moves on.

Algorithm.

Backfitting.

1. Initialize $\hat{\boldsymbol{\beta}}$ by least squares of $\mathbf{y}$ on $\mathbf{X}$, and $\hat{\mathbf{f}}_j = \mathbf{0}$ for $j = 1, \dots, q$.
2. For $j = 1, \dots, q$ in turn, set $\mathbf{r}_j = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}} - \sum_{k \neq j} \hat{\mathbf{f}}_k$ and $\hat{\mathbf{f}}_j \leftarrow \mathbf{S}_j \mathbf{r}_j$.
3. Update $\hat{\boldsymbol{\beta}} \leftarrow (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \sum_k \hat{\mathbf{f}}_k)$, and return to step 2 until the largest change in any $\hat{\mathbf{f}}_j$ is below a tolerance.

Each step is a one-dimensional smoothing problem of the kind Chapter 43 solved; the terms are never assembled into one large matrix; and a smoother *not* of the form Eq. 44.2.1 — a kernel estimate, a monotone fit — drops in unchanged. What is not obvious is that the cycling converges, and to what.

44.2.2 Backfitting is Gauss–Seidel

Theorem 44.2.1. Backfitting

Consider the criterion Eq. 44.1.4 with blocks $\mathbf{Z}_0 = \mathbf{X}$, $\mathbf{P}_0 = \mathbf{0}$ and $\mathbf{Z}_1, \dots, \mathbf{Z}_q$, and write $\mathbf{A}_{jj} = \mathbf{Z}_j^\top \mathbf{Z}_j + \lambda_j \mathbf{P}_j$, assumed nonsingular for every j . Let $\mathbf{A} = \mathbf{Z}^\top \mathbf{Z} + \mathbf{P}$ as in Eq. 44.1.6, so that $\mathbf{A}$ is nonnegative definite with diagonal blocks $\mathbf{A}_{jj}$.

- (a) (*Coordinate descent.*) The backfitting update of the j th block is the exact minimizer of Q over $\boldsymbol{\gamma}_j$ with all other blocks held fixed. In particular Q never increases along the iteration.
- (b) (*Gauss–Seidel.*) In terms of coefficients, the update is $\boldsymbol{\gamma}_j \leftarrow \mathbf{A}_{jj}^{-1} \mathbf{Z}_j^\top (\mathbf{y} - \sum_{k \neq j} \mathbf{Z}_k \boldsymbol{\gamma}_k)$, which is one block Gauss–Seidel sweep for the linear system $\mathbf{A}\boldsymbol{\gamma} = \mathbf{Z}^\top \mathbf{y}$. Fix any order of the blocks — the algorithm above takes them in the order $1, \dots, q, 0$ — and write $\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{L}^\top$ with $\mathbf{D}$ the block diagonal and $\mathbf{L}$ the strictly block lower triangular part in that order. One full cycle sends $\boldsymbol{\gamma}$ to $\mathbf{T}\boldsymbol{\gamma} + (\mathbf{D} + \mathbf{L})^{-1} \mathbf{Z}^\top \mathbf{y}$ with

$$\mathbf{T} = \mathbf{I} - (\mathbf{D} + \mathbf{L})^{-1} \mathbf{A}. \quad (44.2.2)$$

- (c) (*Convergence.*) Let $\boldsymbol{\gamma}^{(t)}$ be the iterates from any starting point and let $\hat{\boldsymbol{\gamma}}$ be any solution of $\mathbf{A}\boldsymbol{\gamma} = \mathbf{Z}^\top \mathbf{y}$. Then $\mathbf{A}^{1/2}(\boldsymbol{\gamma}^{(t)} - \hat{\boldsymbol{\gamma}}) \rightarrow \mathbf{0}$ geometrically. Consequently the fitted vector $\mathbf{Z}\boldsymbol{\gamma}^{(t)}$ converges to $\mathbf{Z}\hat{\boldsymbol{\gamma}}$, the criterion $Q(\boldsymbol{\gamma}^{(t)})$ converges to its minimum, and $\boldsymbol{\gamma}^{(t)}$ itself converges to a minimizer of Q , which may depend on the starting point.
- (d) (*Uniqueness.*) If $\mathbf{A}$ is positive definite, the limit is the unique solution $\mathbf{A}^{-1}\mathbf{Z}^\top \mathbf{y}$ of Eq. 44.1.6, and the convergence is geometric in every norm.

Proof.

- (a) As a function of $\boldsymbol{\gamma}_j$ alone, with $\mathbf{r}_j = \mathbf{y} - \sum_{k \neq j} \mathbf{Z}_k \boldsymbol{\gamma}_k$,

$$Q = \|\mathbf{r}_j - \mathbf{Z}_j \boldsymbol{\gamma}_j\|^2 + \lambda_j \boldsymbol{\gamma}_j^\top \mathbf{P}_j \boldsymbol{\gamma}_j + \text{const},$$

strictly convex because $\mathbf{A}_{jj}$ is nonsingular; its gradient vanishes at $\mathbf{A}_{jj} \boldsymbol{\gamma}_j = \mathbf{Z}_j^\top \mathbf{r}_j$, the stated update, and $\mathbf{Z}_j \boldsymbol{\gamma}_j = \mathbf{S}_j \mathbf{r}_j$ by Eq. 44.2.1. Each step minimizes Q in one block, so Q is nonincreasing.

- (b) The j th block row of $\mathbf{A}\boldsymbol{\gamma} = \mathbf{Z}^\top \mathbf{y}$ reads $\mathbf{A}_{jj} \boldsymbol{\gamma}_j + \sum_{k \neq j} \mathbf{Z}_j^\top \mathbf{Z}_k \boldsymbol{\gamma}_k = \mathbf{Z}_j^\top \mathbf{y}$, and solving it for $\boldsymbol{\gamma}_j$ with the other blocks at their current values is the update in (a); taking the *new* values for $k < j$ and the old for $k > j$ is block Gauss–Seidel. Over one full cycle the new iterate satisfies $(\mathbf{D} + \mathbf{L})\boldsymbol{\gamma}^{\text{new}} = -\mathbf{L}^\top \boldsymbol{\gamma}^{\text{old}} + \mathbf{Z}^\top \mathbf{y}$, which rearranges to Eq. 44.2.2.
- (c) Put $\mathbf{E} = \mathbf{D} + \mathbf{L}$, so that $\mathbf{E} + \mathbf{E}^\top - \mathbf{A} = \mathbf{D}$. The error $\mathbf{e}^{(t)} = \boldsymbol{\gamma}^{(t)} - \hat{\boldsymbol{\gamma}}$ obeys $\mathbf{e}^{(t+1)} = \mathbf{T}\mathbf{e}^{(t)}$. For any $\mathbf{e}$, with $\mathbf{v} = \mathbf{E}^{-1} \mathbf{A}\mathbf{e}$ so that $\mathbf{T}\mathbf{e} = \mathbf{e} - \mathbf{v}$,

$$\mathbf{e}^\top \mathbf{A}\mathbf{e} - (\mathbf{T}\mathbf{e})^\top \mathbf{A}(\mathbf{T}\mathbf{e}) = 2\mathbf{v}^\top \mathbf{A}\mathbf{e} - \mathbf{v}^\top \mathbf{A}\mathbf{v} = 2\mathbf{v}^\top \mathbf{E}\mathbf{v} - \mathbf{v}^\top \mathbf{A}\mathbf{v} = \mathbf{v}^\top (\mathbf{E} + \mathbf{E}^\top - \mathbf{A})\mathbf{v} = \mathbf{v}^\top \mathbf{D}\mathbf{v}, \quad (44.2.3)$$

using $\mathbf{A}\mathbf{e} = \mathbf{E}\mathbf{v}$ and the symmetry of $\mathbf{A}$. The matrix $\mathbf{D}$ is positive definite, so the $\mathbf{A}$ -seminorm of the error never increases, and it decreases strictly unless $\mathbf{v} = \mathbf{0}$, that is unless $\mathbf{A}\mathbf{e} = \mathbf{0}$.

For geometric decay, set $\mathbf{u} = \mathbf{A}^{1/2} \mathbf{e}$ and $\mathbf{N} = \mathbf{I} - \mathbf{A}^{1/2} \mathbf{E}^{-1} \mathbf{A}^{1/2}$, so that $\mathbf{A}^{1/2} \mathbf{T}\mathbf{e} = \mathbf{N}\mathbf{u}$. In terms of $\mathbf{u}$, Eq. 44.2.3 reads, for every $\mathbf{u} \in \mathbb{R}^d$,

$$\|\mathbf{u}\|^2 - \|\mathbf{N}\mathbf{u}\|^2 = \mathbf{v}^\top \mathbf{D}\mathbf{v}, \quad \mathbf{v} = \mathbf{E}^{-1} \mathbf{A}^{1/2} \mathbf{u}. \quad (44.2.4)$$

Now $\mathcal{C}(\mathbf{A}^{1/2})$ is invariant under $\mathbf{N}$, since $\mathbf{N}\mathbf{u} = \mathbf{u} - \mathbf{A}^{1/2}(\mathbf{E}^{-1} \mathbf{A}^{1/2} \mathbf{u})$, and on that subspace $\mathbf{v} = \mathbf{0}$ forces $\mathbf{A}^{1/2} \mathbf{u} = \mathbf{0}$, hence $\mathbf{u} = \mathbf{0}$. So Eq. 44.2.4 gives $\|\mathbf{N}\mathbf{u}\| < \|\mathbf{u}\|$ for every nonzero $\mathbf{u} \in \mathcal{C}(\mathbf{A}^{1/2})$, and its unit sphere is compact, so $\rho := \max\{\|\mathbf{N}\mathbf{u}\| : \mathbf{u} \in \mathcal{C}(\mathbf{A}^{1/2}), \|\mathbf{u}\| = 1\} < 1$. As $\mathbf{u}^{(0)} \in \mathcal{C}(\mathbf{A}^{1/2})$ and the subspace is invariant, $\|\mathbf{A}^{1/2} \mathbf{e}^{(t)}\| \leq \rho^t \|\mathbf{A}^{1/2} \mathbf{e}^{(0)}\|$.

Finally, $\|\mathbf{A}^{1/2} \mathbf{e}\|^2 = \mathbf{e}^\top \mathbf{A}\mathbf{e} = \|\mathbf{Z}\mathbf{e}\|^2 + \mathbf{e}^\top \mathbf{P}\mathbf{e}$, so both terms tend to zero, which gives $\mathbf{Z}\boldsymbol{\gamma}^{(t)} \rightarrow \mathbf{Z}\hat{\boldsymbol{\gamma}}$ and $Q(\boldsymbol{\gamma}^{(t)}) \rightarrow Q(\hat{\boldsymbol{\gamma}})$. For the coefficients themselves,

$$\boldsymbol{\gamma}^{(t+1)} - \boldsymbol{\gamma}^{(t)} = \mathbf{T}\mathbf{e}^{(t)} - \mathbf{e}^{(t)} = -\mathbf{E}^{-1} \mathbf{A}^{1/2} (\mathbf{A}^{1/2} \mathbf{e}^{(t)}),$$

whose norm is at most $\|\mathbf{E}^{-1} \mathbf{A}^{1/2}\| \rho^t \|\mathbf{A}^{1/2} \mathbf{e}^{(0)}\|$. These increments are summable, so $\boldsymbol{\gamma}^{(t)}$ converges to a point $\boldsymbol{\gamma}^\infty$ with $\mathbf{A}\boldsymbol{\gamma}^\infty = \mathbf{Z}^\top \mathbf{y}$: a minimizer of Q .

- (d) If $\mathbf{A}$ is positive definite then $\mathbf{A}^{1/2}$ is nonsingular, $\mathcal{C}(\mathbf{A}^{1/2}) = \mathbb{R}^d$, and the argument above gives $\|\mathbf{e}^{(t)}\| \leq c \rho^t \|\mathbf{e}^{(0)}\|$ for a constant c depending only on the condition number of $\mathbf{A}$. ■

The unpenalized case is the picture [Chapter 6](#) draws.

Corollary 44.2.2. Backfitting as alternating projection

If $\lambda_j = 0$ for every j and each $\mathbf{Z}_j$ has full column rank, then each $\mathbf{S}_j$ is the orthogonal projection onto $\mathcal{V}_j = \mathcal{C}(\mathbf{Z}_j)$ by Theorem 6.3.3, and backfitting converges to the orthogonal projection of $\mathbf{y}$ onto $\mathcal{V}_0 + \mathcal{V}_1 + \cdots + \mathcal{V}_q$. The fitted components are unique iff the only way to write $\mathbf{0}$ as $\sum_j \mathbf{v}_j$ with $\mathbf{v}_j \in \mathcal{V}_j$ is with every $\mathbf{v}_j = \mathbf{0}$.

Proof. With $\mathbf{P} = \mathbf{0}$ the criterion is $\|\mathbf{y} - \mathbf{Z}\boldsymbol{\gamma}\|^2$ and $\mathbf{A} = \mathbf{Z}^\top \mathbf{Z}$, nonnegative definite with nonsingular diagonal blocks, which is what [Theorem 44.2.1\(c\)](#) needs; its conclusion is $\mathbf{Z}\boldsymbol{\gamma}^{(t)} \rightarrow \mathbf{Z}\hat{\boldsymbol{\gamma}}$, and $\mathbf{Z}\hat{\boldsymbol{\gamma}} = \mathbf{M}\mathbf{y}$ is the projection onto $\mathcal{C}(\mathbf{Z}) = \sum_j \mathcal{V}_j$ by Theorem 6.4.1. The components are the coordinates of that projection in the sum, unique precisely when the sum is direct, which is the stated condition. ■

44.2.3 What it costs, and when to use it

Example. Backfitting on two P-spline terms

Two covariates on $[0, 1]$ come from a bivariate normal pair squashed through a hyperbolic tangent, so one correlation ρ controls their dependence; the mean is $3 + f_1(z_1) + f_2(z_2)$, cubic-like in the first component and a bump in the second, with normal errors of standard deviation 0.4. With $n = 400$, 15 cubic B-splines per term, second-difference penalties and $\lambda_1 = \lambda_2 = 8.0$, backfitting and one direct solve of Eq. 44.1.6 agree to 9.4×10^{-14} in the fitted values, as they must, using 11.64 effective degrees of freedom: 1 for the intercept, 5.34 and 5.30 for the two components.

The work is not shared. At $\rho = 0.0$ the backfitting change falls below 10^{-12} after 9 cycles, shrinking by a factor of about 0.015 per cycle; at $\rho = 0.5$ it takes 21 cycles and the factor is 0.240; at $\rho = 0.9$ it takes 121 cycles and the factor is 0.674. [Figure 44.2.1](#) shows the fitted components and the three convergence curves, straight on a logarithmic scale as [Theorem 44.2.1\(c\)](#) predicts.

The blocks used throughout the chapter are built once and reused. A P-spline term is a basis, a difference penalty and the constraint of Proposition 44.1.1(c) absorbed into it:

```
import numpy as np
from scipy.interpolate import BSpline

def bspline_basis(x, n_basis=15, degree=3, lo=None, hi=None):
    """n_basis B-splines of the given degree on equally spaced knots covering the
```

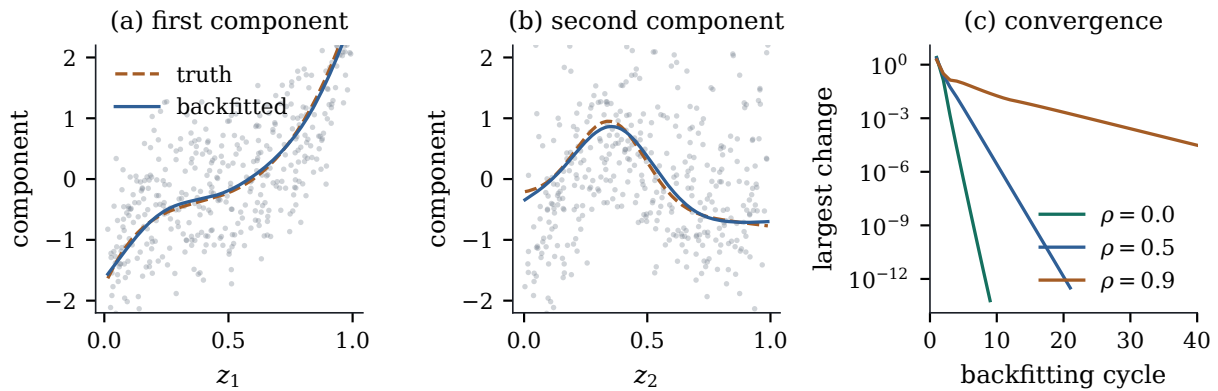


Figure 44.2.1: Backfitting on two P-spline terms. (a) and (b) the fitted components against the truth. (c) the largest change per cycle at three levels of dependence: geometric decay, at a rate that worsens as the terms grow harder to tell apart.

```

data. """
lo = x.min() if lo is None else lo
hi = x.max() if hi is None else hi
inner = np.linspace(lo, hi, n_basis - degree + 1)
h = inner[1] - inner[0]
knots = np.r_[lo - h * np.arange(degree, 0, -1), inner, hi + h * np.arange(1,
degree + 1)]
return BSpline.design_matrix(np.clip(x, lo, hi), knots, degree, extrapolate=
False).toarray()

def difference_penalty(d, order=2):
    """K = D'D for the order-th difference matrix D: the P-spline penalty."""
    D = np.diff(np.eye(d), n=order, axis=0)
    return D.T @ D

def spline_term(x, n_basis=15, order=2, weight=None):
    """A P-spline term: its design, its penalty, and the map from new covariate
    values to
    design rows. The constraint  $\sum_i f(x_i) = 0$  is absorbed into the basis, and
    weight = u turns the term into the varying-coefficient term  $f(x) u$ ."""
    lo, hi = x.min(), x.max()
    B = bspline_basis(x, n_basis, lo=lo, hi=hi)
    Q, _ = np.linalg.qr(B.sum(axis=0)[: , None], mode="complete")
    U = Q[:, 1:] # a basis of {gamma : sum_i f(x_i) = 0}
    Z = B if weight is None else B * np.asarray(weight)[: , None]

    def design(xnew, wnew=None):
        Bn = bspline_basis(np.asarray(xnew, float), n_basis, lo=lo, hi=hi)
        return (Bn if wnew is None else Bn * np.asarray(wnew)[: , None]) @ U

    return Z @ U, U.T @ difference_penalty(n_basis, order) @ U, design

def spline_block(x, n_basis=15, order=2, weight=None):
    """The (design, penalty) pair of spline_term, without the prediction map."""
    return spline_term(x, n_basis, order, weight)[:2]

```

[Run this code online](#)

The data of [Example \(Backfitting on two P-spline terms\)](#) are simulated from two dependent

covariates:

```
def simulate(n, rho, seed=20440):
    """Two dependent covariates on [0, 1] and an additive mean with normal errors.
    """
    rng = np.random.default_rng(seed)
    u = rng.normal(size=(n, 2)) @ np.array([[1.0, rho], [0.0, np.sqrt(1 - rho**2)]])
    z1, z2 = 0.5 * (1 + np.tanh(0.8 * u[:, 0])), 0.5 * (1 + np.tanh(0.8 * u[:, 1]))
    f1 = 16.0 * (z1 - 0.5) ** 3 - 0.6 * np.sin(4.5 * z1)
    f2 = 1.4 * np.exp(-25 * (z2 - 0.35) ** 2) - 0.5 * z2
    f1, f2 = f1 - f1.mean(), f2 - f2.mean()
    y = 3.0 + f1 + f2 + rng.normal(scale=0.4, size=n)
    return z1, z2, f1, f2, y
```

[Run this code online](#)

Backfitting is a dozen lines, and the direct solve another dozen:

```
def backfit(y, blocks, lambdas, tol=1e-12, maxit=600):
    """Backfitting: cycle over the terms, refitting each to the current partial
    residual."""
    A = [Z.T @ Z + lam * K for (Z, K), lam in zip(blocks, lambdas)]
    f = [np.zeros_like(y) for _ in blocks]
    b0, history = y.mean(), []
    for _ in range(maxit):
        change = 0.0
        for j, (Z, _) in enumerate(blocks):
            r = y - b0 - sum(f[k] for k in range(len(blocks)) if k != j)
            new = Z @ np.linalg.solve(A[j], Z.T @ r) # smooth the partial
            residual
            change = max(change, np.abs(new - f[j]).max())
            f[j] = new
        history.append(change)
        if change < tol:
            break
    return b0, f, np.array(history)

def direct_fit(y, blocks, lambdas):
    """One solve of the penalized normal equations, with an intercept column in
    front."""
    Z = np.column_stack([np.ones(len(y))] + [Zj for Zj, _ in blocks])
    K = np.zeros((Z.shape[1], Z.shape[1]))
    start = 1
    for (_, Kj), lam in zip(blocks, lambdas):
        d = Kj.shape[0]
        K[start:start + d, start:start + d] = lam * Kj
        start += d
    C = np.linalg.solve(Z.T @ Z + K, Z.T) # gamma = C y
    edf = [np.trace(Zj @ C[s:s + Zj.shape[1]]) for Zj, s in
            zip([Zj for Zj, _ in blocks], np.cumsum([1] + [Zj.shape[1] for Zj, _ in
            blocks]))]
    return Z @ (C @ y), np.trace(Z @ C), edf
```

[Run this code online](#)

The two agree, and Proposition 44.1.2 falls out of the direct solve:

```
n, lam = 400, 8.0
```

```

z1, z2, f1_true, f2_true, y = simulate(n, rho=0.5)
blocks = [spline_block(z1), spline_block(z2)]
lambdas = [lam, lam]

b0, f, history = backfit(y, blocks, lambdas)
fitted, edf_total, edf = direct_fit(y, blocks, lambdas)
print("cycles:", len(history))
print("largest disagreement:", np.abs(fitted - (b0 + f[0] + f[1])).max())
print(f"edf {edf_total:.3f} = 1 + {edf[0]:.3f} + {edf[1]:.3f}")

```

[Run this code online](#)

The direct solve costs $O(nd^2 + d^3)$ and hands over $\mathbf{A}^{-1}$, hence Proposition 44.1.2 and the covariances, free. Backfitting costs $O(\sum_j nd_j^2)$ a cycle, never forms a $d \times d$ matrix and accepts smoothers no penalty produces, but delivers only the fit and takes more cycles the more the terms depend on one another. Modern software solves directly with sparse factorizations; backfitting survives where d is enormous and where the terms are unusual. It is also close kin to the boosting algorithms of Chapter 30, which fit a block to the current residual as backfitting does but damp the update and pick the block greedily instead of cycling (Theorem 30.4.1).

44.2.4 Concurvity

The rate in Example (Backfitting on two P-spline terms) worsens with ρ for the reason collinearity hurts a linear model: two terms that can each explain the same feature compete for it. Here the competition is between *function spaces*, not columns, and it has its own name.

Definition 44.2.1. Concurvity

Let $\mathcal{V}_0 = \mathcal{C}(\mathbf{X})$ and $\mathcal{V}_j = \mathcal{C}(\mathbf{Z}_j)$ be the model spaces of the terms, all centred so that $\mathcal{V}_j \subseteq \mathbf{1}^\perp$ for $j \geq 1$. The terms exhibit **exact concurvity** if there exist $\mathbf{v}_j \in \mathcal{V}_j$, not all zero, with $\sum_{j \geq 0} \mathbf{v}_j = \mathbf{0}$. The **concurvity index** of term j is

$$\kappa_j = \max \left\{ \frac{\|\mathbf{M}_{-j} \mathbf{v}\|^2}{\|\mathbf{v}\|^2} : \mathbf{v} \in \mathcal{V}_j, \mathbf{v} \neq \mathbf{0} \right\} \in [0, 1], \quad (44.2.5)$$

where $\mathbf{M}_{-j}$ is the orthogonal projection onto $\sum_{k \neq j} \mathcal{V}_k$. Equivalently κ_j is the largest squared canonical correlation between $\mathcal{V}_j$ and $\sum_{k \neq j} \mathcal{V}_k$, and $\kappa_j = 1$ iff there is exact concurvity involving term j .

Proposition 44.2.3. What concurvity does

With the notation of Definition 44.2.1 and $\lambda_j = 0$ for all j :

- (a) $\mathbf{A} = \mathbf{Z}^\top \mathbf{Z}$ is singular iff there is exact concurvity. The fitted values are unaffected — they are the projection onto $\sum_j \mathcal{V}_j$ — but the decomposition into components is not unique.
- (b) Assume in addition that there is no exact concurvity and that each $\mathbf{Z}_j$ has full column rank. Then there is a direction $\mathbf{v} \in \mathcal{V}_j$, $\mathbf{v} \neq \mathbf{0}$, with

$$\text{Var}(\mathbf{v}^\top \hat{\mathbf{f}}_j) = \frac{\sigma^2 \|\mathbf{v}\|^2}{1 - \kappa_j}, \quad (44.2.6)$$

against the value $\sigma^2 \|\mathbf{v}\|^2$ that the same functional would have if $\mathcal{V}_j$ were orthogonal to the other spaces. The inflation factor is $1/(1 - \kappa_j)$, and in the single-column case $\mathcal{V}_j = \text{span}\{\mathbf{x}_j\}$ it is the variance inflation factor of Definition 26.2.1, with $\text{Var}(\hat{\beta}_j) = \sigma^2 / \{\|\mathbf{x}_j\|^2(1 - \kappa_j)\}$.

- (c) (*What a penalty repairs.*) With $\lambda_j > 0$ for every j and $\mathcal{N}(\mathbf{Z}_j) \cap \mathcal{N}(\mathbf{P}_j) = \{\mathbf{0}\}$, the matrix $\mathbf{A}$ is nonsingular and the fitted components are unique, whatever κ_j may be.

Proof.

- (a) $\mathbf{Z}^\top \mathbf{Z}$ is singular iff $\mathbf{Z}\boldsymbol{\gamma} = \mathbf{0}$ for some $\boldsymbol{\gamma} \neq \mathbf{0}$, that is iff the vectors $\mathbf{v}_j = \mathbf{Z}_j \boldsymbol{\gamma}_j$ sum to zero without all being zero (if some $\boldsymbol{\gamma}_j \neq \mathbf{0}$ gives $\mathbf{v}_j = \mathbf{0}$, then $\mathbf{Z}_j$ itself is rank deficient, a within-term problem that the constraint of Proposition 44.1.1(c) and the choice of basis exclude). The fitted vector is the projection onto $\mathcal{C}(\mathbf{Z})$, which does not depend on the basis (Theorem 6.3.6).
- (b) Put $\mathbf{A}_j = \mathbf{Z}_j^\top \mathbf{Z}_j$, $\tilde{\mathbf{Z}}_j = (\mathbf{I} - \mathbf{M}_{-j})\mathbf{Z}_j$ and $\tilde{\mathbf{A}}_j = \tilde{\mathbf{Z}}_j^\top \tilde{\mathbf{Z}}_j$, both positive definite. Writing $\mathbf{v} = \mathbf{Z}_j \mathbf{c}$, Eq. 44.2.5 says that $1 - \kappa_j$ is the minimum of $\mathbf{c}^\top \tilde{\mathbf{A}}_j \mathbf{c} / \mathbf{c}^\top \mathbf{A}_j \mathbf{c}$, attained at a generalized eigenvector $\mathbf{c}$ satisfying $\tilde{\mathbf{A}}_j \mathbf{c} = (1 - \kappa_j) \mathbf{A}_j \mathbf{c}$ by Corollary 1.7.8. Now Theorem 6.6.2 with $\mathbf{X}_1 = \mathbf{Z}_j$ and $\mathbf{X}_2$ the other blocks gives $\hat{\boldsymbol{\gamma}}_j = \tilde{\mathbf{A}}_j^{-1} \tilde{\mathbf{Z}}_j^\top \mathbf{y}$ and $\text{Cov}(\hat{\boldsymbol{\gamma}}_j) = \sigma^2 \tilde{\mathbf{A}}_j^{-1}$. Since $\mathbf{v}^\top \hat{\mathbf{f}}_j = \mathbf{c}^\top \mathbf{A}_j \hat{\boldsymbol{\gamma}}_j$,

$$\text{Var}(\mathbf{v}^\top \hat{\mathbf{f}}_j) = \sigma^2 \mathbf{c}^\top \mathbf{A}_j \tilde{\mathbf{A}}_j^{-1} \mathbf{A}_j \mathbf{c} = \frac{\sigma^2}{1 - \kappa_j} \mathbf{c}^\top \mathbf{A}_j \mathbf{c} = \frac{\sigma^2 \|\mathbf{v}\|^2}{1 - \kappa_j},$$

because the eigenvector relation gives $\tilde{\mathbf{A}}_j^{-1} \mathbf{A}_j \mathbf{c} = \mathbf{c} / (1 - \kappa_j)$. If $\mathcal{V}_j \perp \sum_{k \neq j} \mathcal{V}_k$, then $\tilde{\mathbf{A}}_j = \mathbf{A}_j$, $\kappa_j = 0$ and the variance is $\sigma^2 \|\mathbf{v}\|^2$. For a single column, $\tilde{\mathbf{A}}_j = \|(\mathbf{I} - \mathbf{M}_{-j})\mathbf{x}_j\|^2 = \|\mathbf{x}_j\|^2(1 - \kappa_j)$, which is Exercise (B1).

- (c) Under the stated condition, $\boldsymbol{\gamma}^\top \mathbf{A} \boldsymbol{\gamma} = \|\mathbf{Z}\boldsymbol{\gamma}\|^2 + \boldsymbol{\gamma}^\top \mathbf{P} \boldsymbol{\gamma} = 0$ forces $\mathbf{Z}_j \boldsymbol{\gamma}_j = \mathbf{0}$ and $\mathbf{P}_j \boldsymbol{\gamma}_j = \mathbf{0}$ for every j , hence $\boldsymbol{\gamma} = \mathbf{0}$. ■

Part (b) is what “concurvity is collinearity for function spaces” means precisely: the worst direction inside the j th model space pays exactly the inflation factor $1/(1 - \kappa_j)$, and κ_j is computed from the design alone, before any data are seen.

Remark (A penalty restores uniqueness, not precision). Part (c) says nothing about variances, and the usual informal claim — that they blow up as $\kappa_j \rightarrow 1$ — is not a theorem: with $\lambda_j > 0$ fixed, $\mathbf{A}$ stays nonsingular and the variances of Proposition 44.1.2(c) stay bounded. What the penalty does is take over the identification: the closer κ_j is to one, the less of $\hat{\mathbf{f}}_j$ is settled by the data and the more by $\mathbf{P}_j$. Exact variances cost one matrix inverse; compute them and κ_j , and report both.

Example. Two covariates, one relation

Let z_1 be uniform on $[0, 1]$ and $z_2 = z_1^{1.5} + \text{noise}$, and fit two P-spline terms with $\lambda = 5.0$ to $n = 300$ observations. The design is fixed and the criterion quadratic, so Proposition 44.1.2(c) gives the exact standard deviation of each component without simulation. As the noise falls

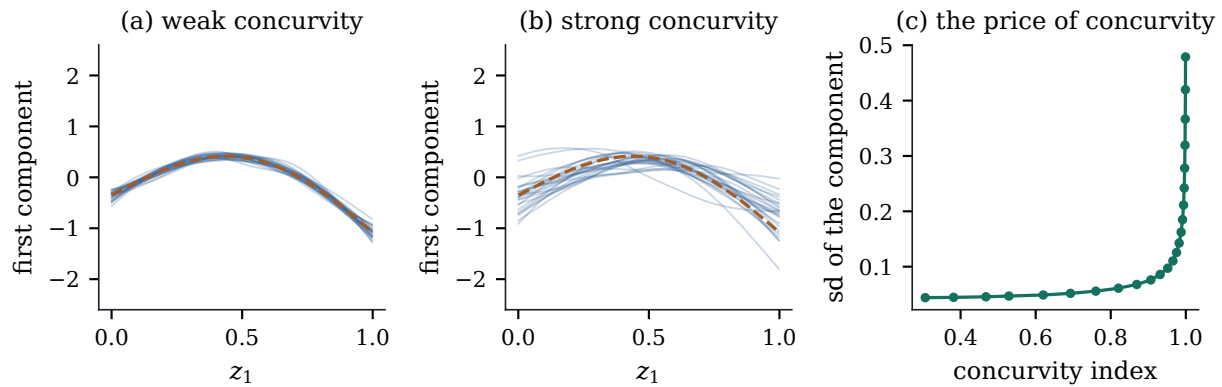


Figure 44.2.2: Concurrency. (a) and (b) twenty-five replicate estimates of the first component (truth dashed) under weak and strong dependence. (c) its exact standard deviation at $z_1 = 0.3$ against the concurrency index Eq. 44.2.5.

from 0.30 through 0.10 to 0.03, the concurrency index of the first term climbs from 0.609 to 0.933 to 0.994, and the standard deviation of the first component at $z_1 = 0.3$ climbs from 0.049 to 0.087 to 0.207, an inflation of 4.2; over the same range the standard deviation of the *fitted mean* there moves only from 0.094 to 0.080. The fit is fine; its attribution to the two covariates is not.

```
def design(n, noise, seed=44012):
    """z2 is a noisy monotone function of z1: small noise means strong concurrency.
    """
    rng = np.random.default_rng(seed)
    z1 = rng.uniform(0, 1, n)
    z2 = np.clip(z1*1.5 + noise * rng.normal(size=n), 0.001, 0.999)
    return z1, z2

def concurrency(Z1, Z2):
    """The largest squared canonical correlation between the two column spaces."""
    Q1, _ = np.linalg.qr(Z1)
    Q2, _ = np.linalg.qr(Z2)
    return np.linalg.svd(Q1.T @ Q2, compute_uv=False)[0] ** 2
```

[Run this code online](#)

Warning. Concurrency is harder to spot than collinearity, because it is invisible in pairwise scatterplots and correlations: two covariates can be uncorrelated and still have model spaces that nearly intersect, since κ_j involves *all* smooth functions of each, not just the linear ones (Exercise B3). Report κ_j alongside the components, and treat a value near one as a warning that the individual curves, and any story told about them, are not identified by the data even though the predictions are.

44.2.5 Exercises

44.2.5.1 A. Check your understanding

Exercise. A1

Run backfitting with $q = 1$. How many cycles does it take, and why?

Solution. One. With a single block the update sets $\boldsymbol{\gamma}_1 = \mathbf{A}_{11}^{-1} \mathbf{Z}_1^\top \mathbf{y}$, which already solves Eq. 44.1.6; a second cycle reproduces it. In Theorem 44.2.1(b), $\mathbf{L} = \mathbf{0}$, so $\mathbf{T} = \mathbf{I} - \mathbf{A}^{-1} \mathbf{A} = \mathbf{0}$.

44.2.5.2 B. Practice

Exercise. B1

For $q = 2$ with no parametric part, show that the Gauss–Seidel iteration matrix $\mathbf{T}$ of Eq. 44.2.2 has spectral radius equal to the largest eigenvalue of $\mathbf{A}_{11}^{-1} \mathbf{A}_{12} \mathbf{A}_{22}^{-1} \mathbf{A}_{21}$, and that for unpenalized one-column blocks this is the squared correlation between the two columns.

Solution. With $\mathbf{D} + \mathbf{L} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{0} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}$ and $\mathbf{L}^\top = \begin{pmatrix} \mathbf{0} & \mathbf{A}_{12} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}$,

$$\mathbf{T} = -(\mathbf{D} + \mathbf{L})^{-1} \mathbf{L}^\top = - \begin{pmatrix} \mathbf{0} & \mathbf{A}_{11}^{-1} \mathbf{A}_{12} \\ \mathbf{0} & -\mathbf{A}_{22}^{-1} \mathbf{A}_{21} \mathbf{A}_{11}^{-1} \mathbf{A}_{12} \end{pmatrix},$$

block triangular with eigenvalues 0 and those of $\mathbf{A}_{22}^{-1} \mathbf{A}_{21} \mathbf{A}_{11}^{-1} \mathbf{A}_{12}$, whose nonzero eigenvalues are those of $\mathbf{A}_{11}^{-1} \mathbf{A}_{12} \mathbf{A}_{22}^{-1} \mathbf{A}_{21}$. With one column per block and no penalty this is $(\mathbf{x}_1^\top \mathbf{x}_2)^2 / (\|\mathbf{x}_1\|^2 \|\mathbf{x}_2\|^2)$, the squared correlation of centred columns.

Exercise. B2

Modified backfitting. Suppose the smoothers $\mathbf{S}_j$ are not of the form Eq. 44.2.1 but are symmetric with eigenvalues in $[0, 1]$ and satisfy $\mathbf{S}_j \mathbf{1} = \mathbf{1}$. Show that replacing $\mathbf{S}_j$ by $\mathbf{S}_j^c = (\mathbf{I} - n^{-1} \mathbf{1} \mathbf{1}^\top) \mathbf{S}_j$ makes every fitted component sum to zero at each step, and that $\mathbf{A}$ in Theorem 44.2.1 is then replaced by a matrix with the same structure acting on $\mathbf{1}^\perp$.

Exercise. B3

Let z_1 be uniform on $[-1, 1]$ and $z_2 = z_1^2$. Show that $\text{Cov}(z_1, z_2) = 0$, so the two covariates are uncorrelated, and yet the two model spaces of quadratic functions exhibit exact concavity. What does this say about diagnosing concavity by a correlation matrix?

Solution. $E(Z_1 Z_1^2) = E(Z_1^3) = 0 = E(Z_1) E(Z_1^2)$, so the covariance is zero. But $g_1(z_1) = z_1^2$ lies in the first space, $g_2(z_2) = z_2$ in the second, and $g_1(z_{i1}) - g_2(z_{i2}) = 0$ for every i : a nontrivial additive relation, so $\kappa_1 = \kappa_2 = 1$. A correlation sees only the linear part of the relation between covariates; concavity sees all of it, so it must be measured on the model spaces.

44.2.5.3 C. Going deeper

Exercise. C1

Population backfitting. Let Z_1, Z_2 be random with $E(Y^2) < \infty$, and consider minimizing $E\{Y - g_1(Z_1) - g_2(Z_2)\}^2$ over centred square-integrable g_1, g_2 . Show that the minimizers satisfy $g_1(z) = E\{Y - g_2(Z_2) \mid Z_1 = z\} - c$ and symmetrically, and that the sample algorithm of this section is the empirical version with conditional expectation replaced by a smoother.